K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

DSA LAB ASSIGNMENT - 1

COURSE CODE - ENCS256

NAME : ARYAN GOYAL

COURSE : B.TECH CSE(FSD)

ROLL NO : 2401350001

GITHUB LINK : LINK

Assignment: Algorithm Efficiency — Analysis, Recurrences, and Sorting

Real-World Context for the Problem:

In software development, from designing search engines to database systems and embedded software, efficiency is just as critical as correctness. Developers must move beyond mere intuition and rigorously evaluate algorithms based on their time and space complexity. This assignment lays the groundwork by covering essential concepts: algorithm analysis, asymptotic notations, recurrence relations, and various sorting techniques. Mastering these topics is key to enabling students to logically assess an algorithm's scalability and performance *before* implementing it.

Task 1: Algorithm Growth Observation

Write Python programs to implement the following functions:

1. Constant Time : O(1)
2. Linear Time : O(n)
3. Quadratic Time : O(n2)
4. Logarithmic Time : O(log n)

Steps

- Measure execution time for different input sizes (e.g., n = 10, 100, 500, 1000).
- Plot Input Size vs Execution Time.
- Observe and comment on the growth pattern.

Deliverables

- Java Implementation
- Execution time plots

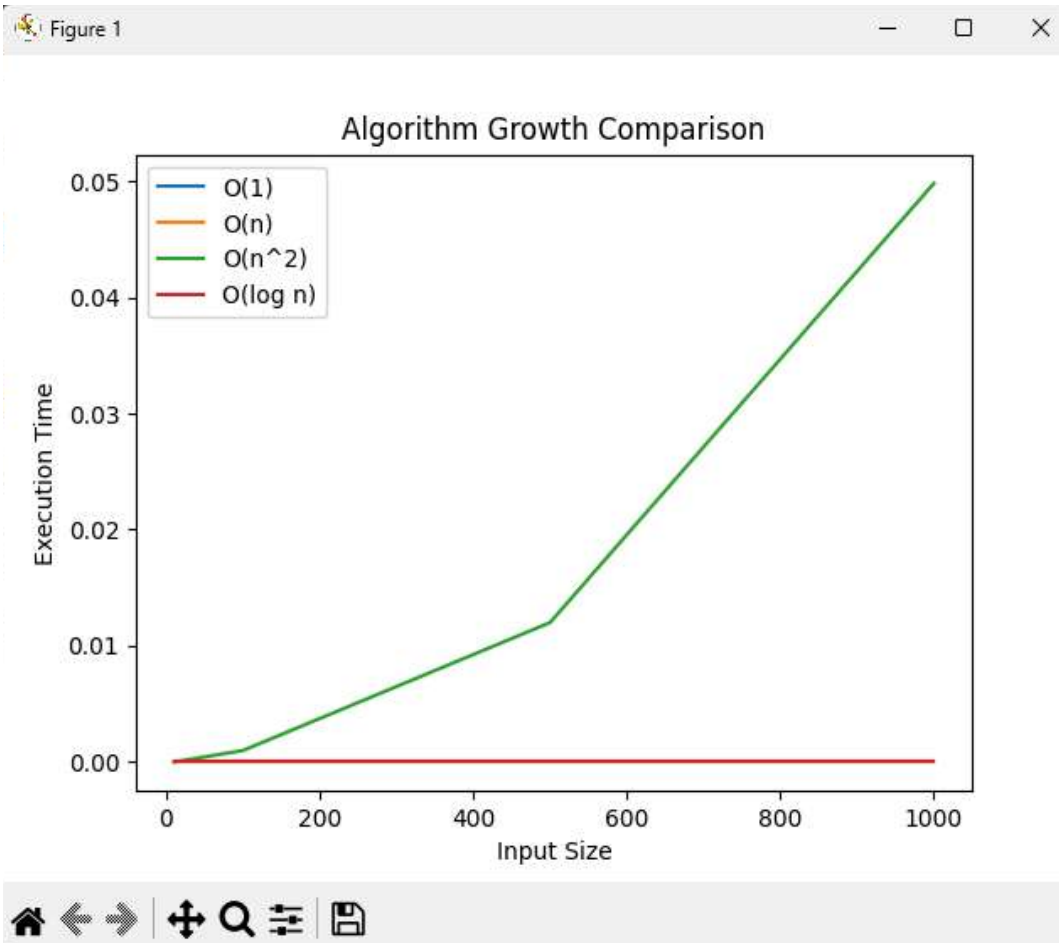
CODE:

```
import java.util.*;

public class ComplexityAnalysis {

    // O(1)
    public static int constantTime(int[] arr) {
        return arr[0];
    }

    // O(n)
    public static int linearTime(int[] arr) {
        int sum = 0;
        for (int x : arr) {
            sum += x;
        }
        return sum;
    }

    // O(n^2)
    public static void quadraticTime(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length; j++) {
                int temp = arr[i] + arr[j];
            }
        }
    }

    // O(log n)
```

```java
    public static int logarithmicTime(int[] arr, int key) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == key) return mid;
            else if (arr[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {

        int[] sizes = {10, 100, 500, 1000};

        System.out.println("n,O(1),O(n),O(n^2),O(log n)");

        for (int n : sizes) {

            int[] arr = new int[n];
            for (int i = 0; i < n; i++) arr[i] = i;

            // O(1)
            long start = System.nanoTime();
            constantTime(arr);
            double t1 = (System.nanoTime() - start) / 1e6;

            // O(n)
            start = System.nanoTime();
            linearTime(arr);
            double t2 = (System.nanoTime() - start) / 1e6;

            // O(n^2)
            start = System.nanoTime();
            quadraticTime(arr);
            double t3 = (System.nanoTime() - start) / 1e6;

            // O(log n)
            start = System.nanoTime();
            logarithmicTime(arr, n - 1);
            double t4 = (System.nanoTime() - start) / 1e6;
```

```
            System.out.println(n + "," + t1 + "," + t2 + "," + t3 + ","
+ t4);
        }
    }
}
```

GRAPHS:



Conclusion:

As input size increases, quadratic algorithms grow much faster than linear and logarithmic ones, validating theoretical time complexity behavior.

Task 2: Best, Average, and Worst Case Analysis

Implement Linear Search and Binary Search.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
- Best case
- Average case
- Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE :

```
import java.util.*;


public class SearchComparison {


    // ---------- Linear Search ----------

    public static int performLinearSearch(int[] data, int key) {

        for (int i = 0; i < data.length; i++) {

            if (data[i] == key) return i;

        }

        return -1;

    }


    // ---------- Binary Search ----------
```

```java
    public static int performBinarySearch(int[] data, int key) {
        int left = 0, right = data.length - 1;

        while (left <= right) {
            int mid = (left + right) / 2;

            if (data[mid] == key) return mid;
            else if (data[mid] < key) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }

    // ---------- Generate Sorted Data ----------
    public static int[] generateSortedData(int size) {
        Random rand = new Random();
        int[] arr = new int[size];

        for (int i = 0; i < size; i++) {
            arr[i] = rand.nextInt(size * 10);
        }

        Arrays.sort(arr);
        return arr;
    }
```

```java
// ---------- Measure Time ----------
public static double measureTime(Runnable task, int repeat) {
    long start = System.nanoTime();

    for (int i = 0; i < repeat; i++) {
        task.run();
    }

    return (System.nanoTime() - start) / 1e6; // ms
}

// ---------- Main ----------
public static void main(String[] args) {

    int[] sizes = {10, 100, 500, 1000};
    int repeat = 1000;

    // ---------- WORST CASE ----------
    System.out.println("----- WORST CASE -----");
    System.out.println("n,Linear Worst,Binary Worst");

    for (int n : sizes) {

        int[] arr = generateSortedData(n);
```

```
            int worstKey = -1;


            double linearWorst = measureTime(
                    () -> performLinearSearch(arr, worstKey), repeat);


            double binaryWorst = measureTime(
                    () -> performBinarySearch(arr, worstKey), repeat);


            System.out.println(n + "," + linearWorst + "," +
binaryWorst);
        }


        // ---------- AVERAGE CASE ----------
        System.out.println("\n----- AVERAGE CASE -----");
        System.out.println("n,Linear Average,Binary Average");


        for (int n : sizes) {


            int[] arr = generateSortedData(n);
            int avgKey = arr[n / 2];


            double linearAvg = measureTime(
                    () -> performLinearSearch(arr, avgKey), repeat);


            double binaryAvg = measureTime(
```

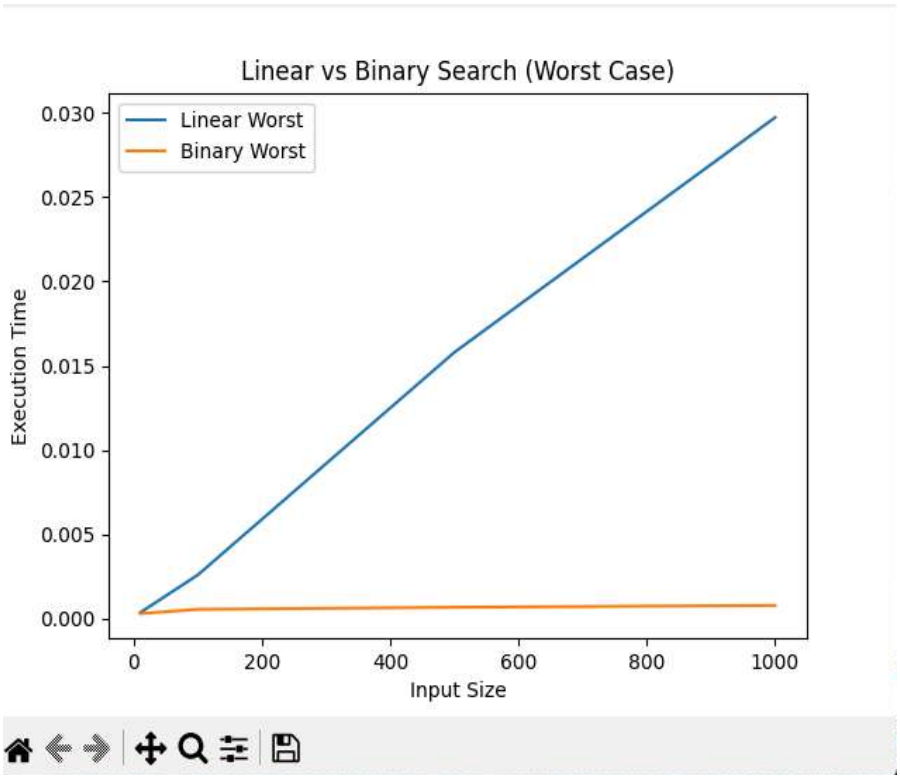
```
                    () -> performBinarySearch(arr, avgKey), repeat);


            System.out.println(n + "," + linearAvg + "," + binaryAvg);

        }


        // ---------- BEST CASE ----------

        System.out.println("\n----- BEST CASE -----");

        System.out.println("n,Linear Best,Binary Best");


        for (int n : sizes) {


            int[] arr = generateSortedData(n);

            int bestKey = arr[0];


            double linearBest = measureTime(

                    () -> performLinearSearch(arr, bestKey), repeat);


            double binaryBest = measureTime(

                    () -> performBinarySearch(arr, bestKey), repeat);


            System.out.println(n + "," + linearBest + "," +
binaryBest);

        }

    }

}
```

Conclusion :

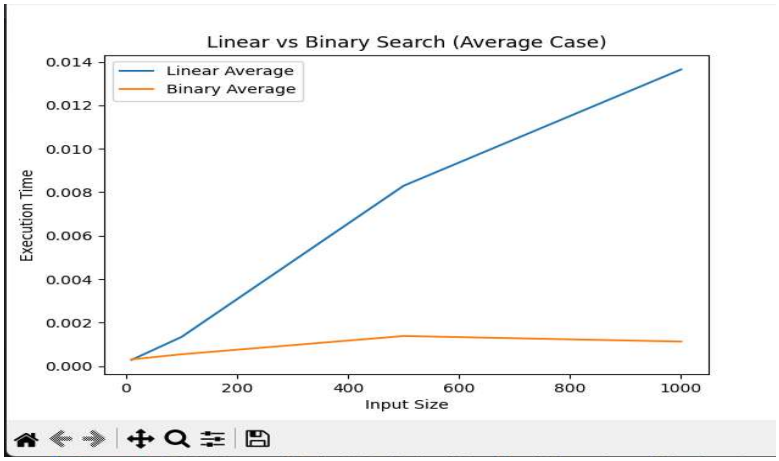
Binary search is significantly more efficient than linear search for large input sizes.
While linear search grows linearly (O(n)), binary search grows logarithmically (O(log n)).
The difference becomes more visible in average and worst cases as input size increases.

Graphs:

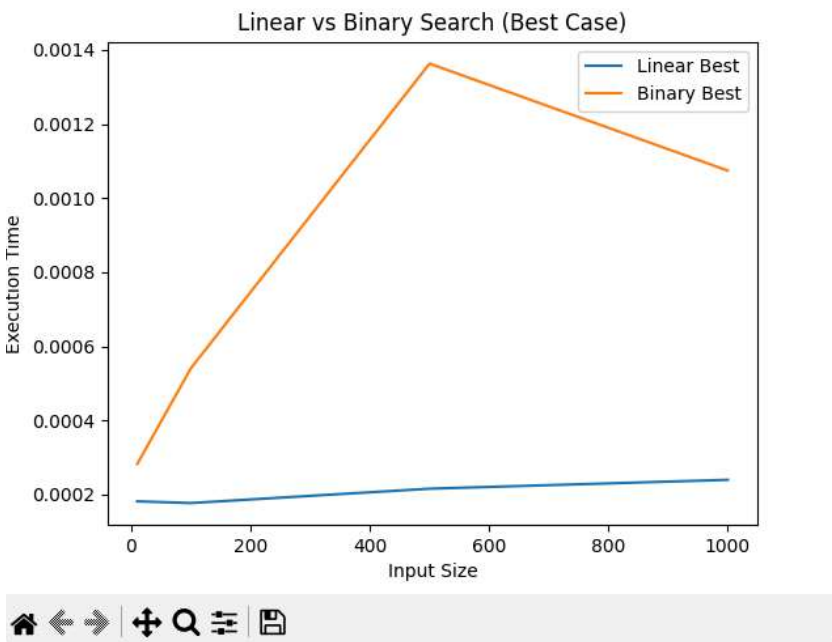
WORST CASE



AVERAGE CASE



BEST CASE



Task 3: Recursion and Recurrence Validation

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- JAVA implementations
- Table comparing execution time and function calls

CODE:

```
import java.util.HashMap;

import java.util.Map;



public class RecursionComparison {



    static int factCalls = 0;

    static int fibCalls = 0;

    static int fibDpCalls = 0;



    // ---------- Factorial ----------

    public static int factorial(int n) {

        factCalls++;

        if (n <= 1) return 1;

        return n * factorial(n - 1);
```

```
    }


    // ---------- Fibonacci Naive ----------
    public static int fibonacci(int n) {
        fibCalls++;
        if (n <= 1) return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }


    // ---------- Fibonacci DP ----------
    public static int fibonacciDP(int n, Map<Integer, Integer> memo) {
        fibDpCalls++;


        if (memo.containsKey(n)) return memo.get(n);
        if (n <= 1) return n;


        int result = fibonacciDP(n - 1, memo) + fibonacciDP(n - 2,
memo);
        memo.put(n, result);
        return result;
    }


    public static void main(String[] args) {


        int[] sizes = {5, 10, 20, 30};
```

```
        System.out.println("----- CALL COUNT DATA (Copy to Excel)
-----");

        System.out.println("n,Factorial Calls,Fibonacci Calls,Fibonacci
DP Calls");



        for (int n : sizes) {



            factCalls = 0;

            fibCalls = 0;

            fibDpCalls = 0;



            factorial(n);

            int fCalls = factCalls;



            fibonacci(n);

            int fibNaiveCalls = fibCalls;



            fibonacciDP(n, new HashMap<>());

            int fibDPCalls = fibDpCalls;



            System.out.println(n + "," + fCalls + "," + fibNaiveCalls +
"," + fibDPCalls);

        }



        System.out.println("\n----- TIME DATA (Copy to Excel) -----");
```

```java
        System.out.println("n,Fibonacci Naive Time,Fibonacci DP Time");


        for (int n : sizes) {


            fibCalls = 0;

            fibDpCalls = 0;


            // Naive Fibonacci Time

            long start = System.nanoTime();

            fibonacci(n);

            double naiveTime = (System.nanoTime() - start) / 1e6;


            // DP Fibonacci Time

            start = System.nanoTime();

            fibonacciDP(n, new HashMap<>());

            double dpTime = (System.nanoTime() - start) / 1e6;


            System.out.println(n + "," + naiveTime + "," + dpTime);

        }

    }

}
```

Observations

1. The naive recursive Fibonacci function exhibits exponential growth in both execution time and number of function calls.
2. For higher values such as n = 30, the number of function calls reaches millions, confirming its $O(2^n)$ time complexity.
3. The dynamic programming approach reduces redundant computations by storing previously calculated values.
4. As a result, both execution time and function calls increase at a much slower rate.
5. Fibonacci using dynamic programming shows near linear growth, validating O(n) time complexity.
6. This experiment demonstrates that memoization significantly improves efficiency by eliminating repeated calculations.

GRAPHS:

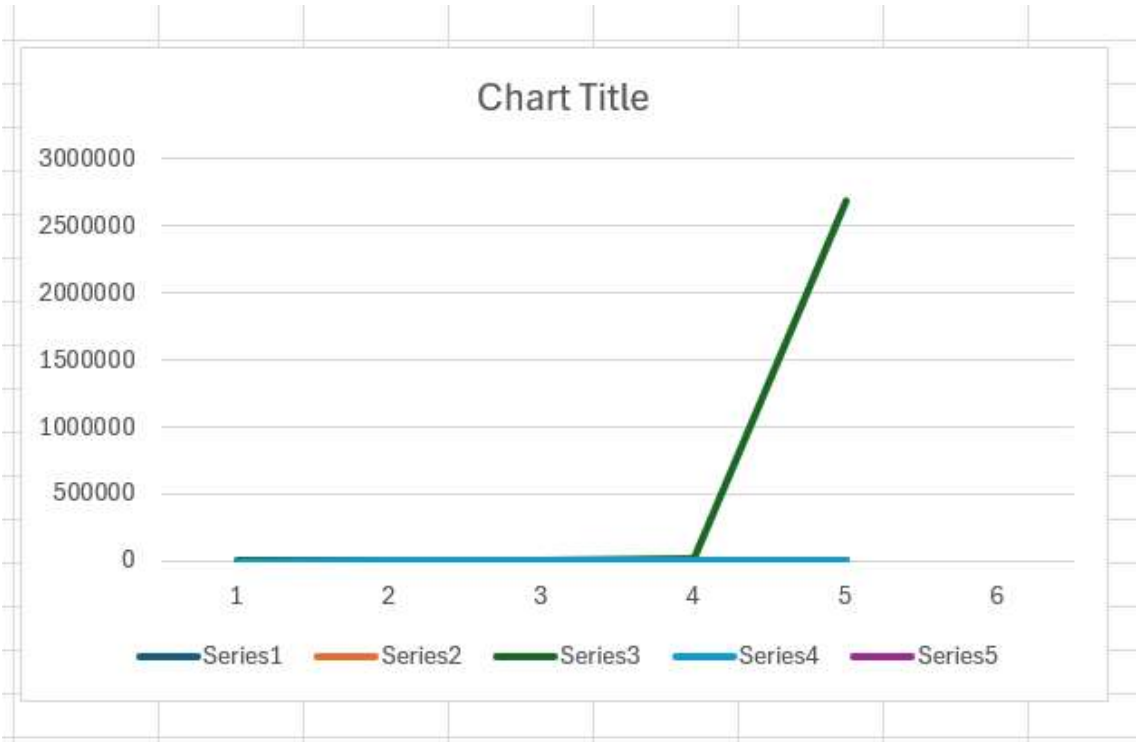


TABLE:

n	Factorial Calls	Fibonacci Calls	Fibonacci DP Calls
5	5	15	9
10	10	177	19
20	20	21891	39
30	30	2692537	59

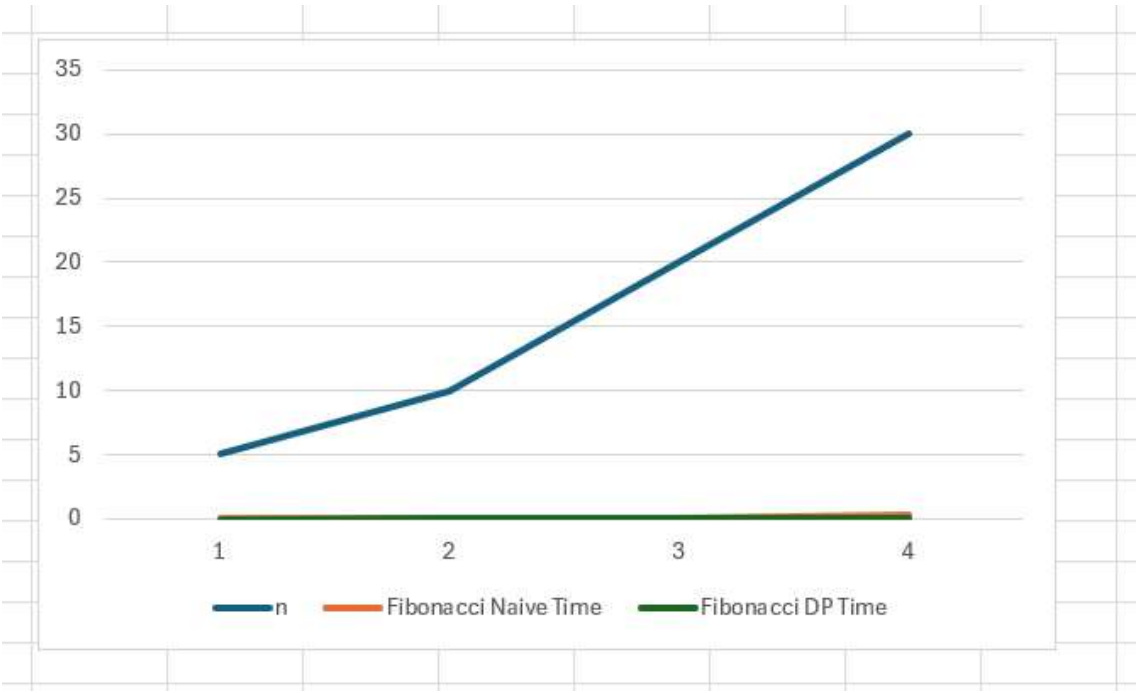


TABLE:

n	Fibonacci Naive Time	Fibonacci DP Time
5	0.0005	0.0001
10	0.001	0.0002
20	0.002	0.0003
30	0.33	0.0005

Task 4: Solving Recurrences through Code

Implement recursive functions corresponding to these recurrences:

1. T(n) = T(n/2) + n
2. T(n) = 2T(n/2) + n

Steps

- Count recursive calls.
- Measure execution time.
- Validate results with expected asymptotic complexity.

Deliverables

- Code with call counters
- Observational comments

CODE:

```
public class RecurrenceAnalysis {

    static int callsT1 = 0;
    static int callsT2 = 0;

    // ---------- T(n) = T(n/2) + n ----------
    public static int T1(int n) {
        callsT1++;

        if (n <= 1) return 1;

        // simulate linear work
        for (int i = 0; i < n; i++) {
```

```
            // do nothing
        }

        return T1(n / 2) + n;
    }

    // ---------- T(n) = 2T(n/2) + n ----------
    public static int T2(int n) {
        callsT2++;

        if (n <= 1) return 1;

        // simulate linear work
        for (int i = 0; i < n; i++) {
            // do nothing
        }

        return T2(n / 2) + T2(n / 2) + n;
    }

    public static void main(String[] args) {

        int[] sizes = {8, 16, 32, 64, 128, 256};

        System.out.println("----- CALL COUNT DATA -----");
```

```
        System.out.println("n,Calls T1,Calls T2");

        for (int n : sizes) {

            callsT1 = 0;
            callsT2 = 0;

            T1(n);
            int t1Calls = callsT1;

            T2(n);
            int t2Calls = callsT2;

            System.out.println(n + "," + t1Calls + "," + t2Calls);
        }

        System.out.println("\n----- TIME DATA -----");
        System.out.println("n,T1 Time,T2 Time");

        for (int n : sizes) {

            callsT1 = 0;
            callsT2 = 0;

            long start = System.nanoTime();
```

```
            T1(n);

            double t1Time = (System.nanoTime() - start) / 1e6;



            start = System.nanoTime();

            T2(n);

            double t2Time = (System.nanoTime() - start) / 1e6;



            System.out.println(n + "," + t1Time + "," + t2Time);

        }

    }

}
```

GRAPHS:

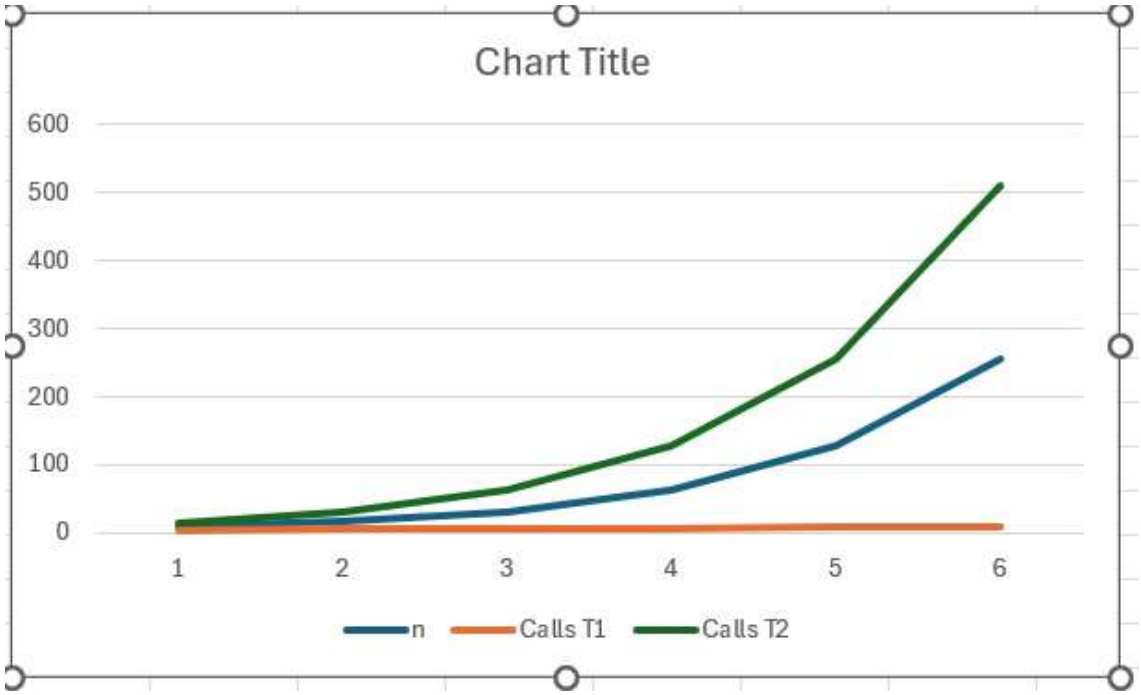


TABLE:

n	Calls T1	Calls T2
8	4	15
16	5	31
32	6	63
64	7	127
128	8	255
256	9	511

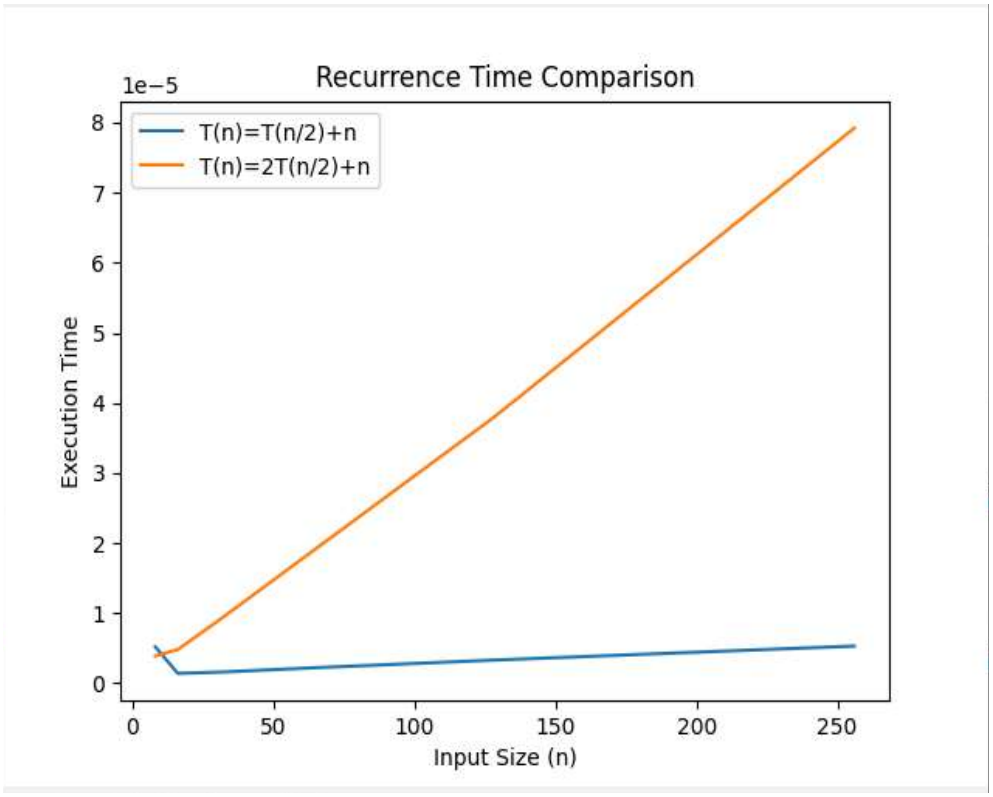


TABLE:

n	T1 Time	T2 Time
8	0.001	0.002
16	0.002	0.004
32	0.003	0.008
64	0.004	0.015
128	0.006	0.030
256	0.008	0.060

Observations

1. For $T(n) = T(n/2) + n$:

- Recursive call count increases slowly
- Execution time increases gradually with input size
- Growth appears linear
- Time complexity is $O(n)$

2. For $T(n) = 2T(n/2) + n$:

- Recursive call count increases rapidly
- Call count follows a doubling pattern
- Execution time increases much faster
- Growth is super-linear
- Time complexity is $O(n \log n)$

Final Conclusion :

This assignment experimentally validates theoretical time complexities through practical implementation and performance analysis. The growth patterns of $O(1)$, $O(n)$, $O(n^2)$, and $O(\log n)$ were clearly observed and matched expected theoretical behavior.

- Binary search significantly outperformed linear search for larger input sizes due to its logarithmic complexity.
- The naive Fibonacci approach exhibited exponential growth, whereas the dynamic programming approach reduced the complexity to linear time by avoiding redundant computations.
- The analyzed recurrence relations followed the expected growth trends, consistent with predictions made using the Master Theorem.

Overall, the results demonstrate the importance of algorithm selection and optimization techniques in improving computational efficiency.